场景营销前端 AI Coding — AI Native 的视觉稿还原

原创 营销&交易技术 大淘宝技术 2026年6月24日 00:26 浙江 2人

传统D2C平台需大量人工配合（图层整理、切图、多状态识别），且视觉还原与业务逻辑脱节。Tarot Pixel 创新提出"不生成代码，让 Coding Agent 自己看懂设计稿"的理念。核心方案是将设计稿转为结构化视觉预览，提供 REST API 让 Agent 按需查询（而非全量推送），形成"实现→比对→修正"闭环。工程层负责精确数据提取与降噪，AI层专注语义理解。优势是无需手动选图层/切图，支持持续修正，人工干预大幅减少。本质是 Agent-Native 设计——工具为 AI 服务，提供精准上下文，减少干扰，让拥有完整项目上下文的 Agent 自主决策实现。

01 引言

上一篇文章讨论了 AI Coding 的核心问题：上下文膨胀、注意力坍塌、人机窗口不对称，以及介绍了塔罗任务规划（Specflow）Agent 如何通过“职责分离”—— 将上下文准备与编码执行拆开——来应对这些问题。

但是有一类任务始终最棘手：**视觉还原**，核心原因也已经在上文指出：

> 单独跑一遍 D2C 平台生成的代码，往往只是个空壳，离“能用”还有很大距离。因为在真实前端开发里，UI 还原从来不是孤立任务，而只是完整功能开发中的一个环节。

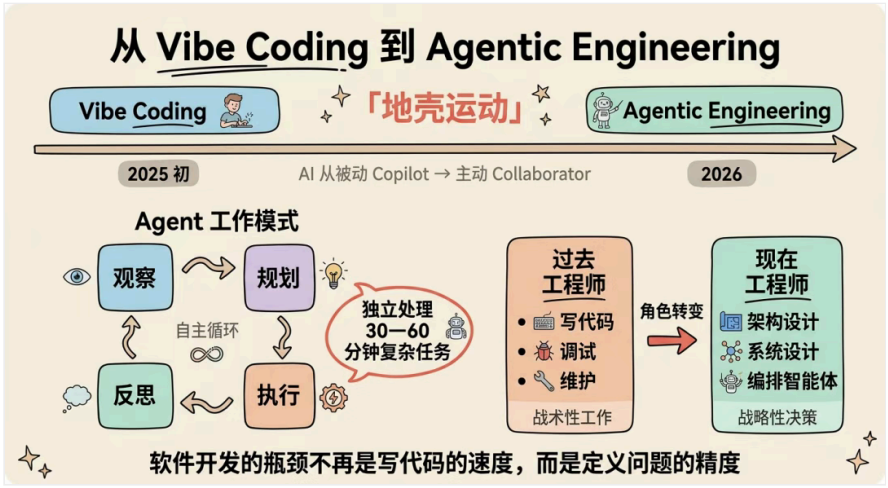
这篇文章要讲的是 Tarot Pixel —— 一个基于 AI Native 思维探索的视觉还原方案，**它不负责生成代码，而是让 Coding Agent 自己看懂设计稿。**

更准确地说，这篇文章讨论的不是“再做一个设计稿转代码的平台”，而是：**当 Coding Agent 已经成为开发主力之后，设计稿应该以什么方式进入它的工作流。**

首先，可以通过这个视频先了解 Tarot Pixel 的使用流程。

02
Agent 时代的工具设计范式

从 Vibe Coding 到 Agentic Engineering



Andrej Karpathy 在 2025 年初提出“vibe coding”这个说法，用来描述一种更依赖直觉、和模型一起快速试错的编程方式。到了今天，他又提到了一个更能概括当前变化的词：agentic engineering。

我觉得这不是换个词，真正变的是软件开发里的分工逻辑。以前 AI 更多停留在补全和局部生成层面，现在它开始能接手一段相对完整的执行过程。于是开发里的稀缺能力，也开始从“把代码写出来”转向“定义问题、表达约束，以及判断结果是否可接受”。

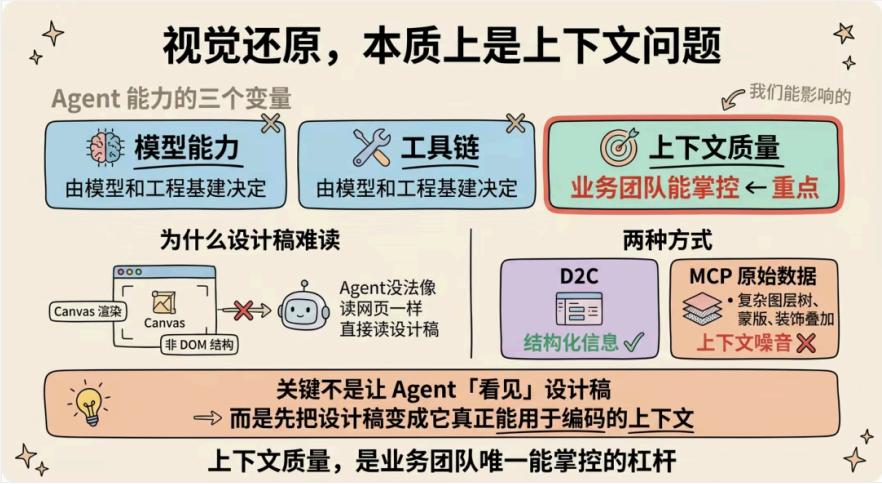
Anthropic 在年初发布的《2026 Agentic Coding Trends Report》也提到了这一点。他们用了“地壳运动”这个比喻，来形容软件开发正在发生的变化。

当下的事实是：

- **第一：Coding Agent 已经不只是生成代码，而是能在 30–60 分钟内独立推进一段完整任务：**读懂代码库、跨文件重构、调用工具链完成构建、测试和调试，再根据结果继续修正。
- **第二：开发者的角色确实在变，但不是人单纯从写代码转去做架构**，而是开始把架构设计、系统设计和一部分判断逻辑转化为 Agent 可执行的能力和约束，让它承担更多过去需要资深工程师亲自推进的工作。
- **第三：Agent 真正的价值，不在于一次性生成一段结果，**而在于它能跑起“观察 → 规划 → 执行 → 反思”的自主循环，在执行中不断修正，把任务持续往前推进。

但即便如此，Anthropic 的研究团队也发现了一个“协作悖论”：尽管工程师已经在大量工作中使用 AI，但真正能完全委托给 AI 的任务比例仍然不高。说到底，Agent 能做多远，不只取决于模型本身，也取决于工具、上下文，以及任务是否足够清晰。

视觉还原的问题，本质上是上下文问题



> 模型能力和工具链，基本都由平台方在持续迭代，比如 Cursor、Claude Code。背后的基建是几亿、几十亿美元级别的投入，业务团队很难真正影响这些底层能力。

但**上下文质量**还是掌握在业务团队手里的变量，而且它正在成为新的核心能力 —— 如何为 Agent 精准地提供它需要的信息，同时过滤掉干扰它的噪声。

如果说上一篇里提到的 Specflow，解决的是“需求理解和任务拆解”这一层的上下文，那前端开发里最难的视觉还原，就是另一类上下文问题。

对 Coding Agent 来说，难点不在代码生成，而在它拿不到足够准确、又适合实现的视觉信息。它可以访问项目代码库，也具备编码和一定的任务推进能力，但视觉信息这一侧一直比较薄弱。

问题在于，设计稿并不像网页那样天然适合被读取。Figma、MasterGo 这类工具本质上是为设计师构建的创作环境，界面基于 Canvas 渲染，不是对代码工具友好的 DOM 结构。对 Coding Agent 来说，它没法像读网页一样直接读设计稿，现在主要还是依赖设计工具提供的 MCP 或插件 API。

传统 D2C 平台其实已经在这个方向上积累了很多能力，比如布局识别、组件匹配、样式提取、图层解析。开发者选中图层后就能拿到一份可用代码。换个角度看，这些能力本质上是在把设计稿转成更适合实现的结构化信息。另一种方式，是通过设计工具的 MCP 直接让 Coding Agent 访问设计稿。但 MCP 返回的往往是设计稿的原始数据：复杂图层树、蒙版、装饰叠加、切图边界、多状态画板……这些信息如果不经过处理就直接丢给 Agent，很多时候只会增加上下文噪音，干扰真正关键的实现判断。

也正因为这个问题，社区里开始出现两条不同的路线。

1. 一条是像 Pencil 这样的 IDE 原生工具，试图绕开 Figma，把视觉设计直接放回代码环境里完成，让视觉结果从一开始就以更接近 W3C、可版本控制、对 Agent 更友好的方式被生产出来；
2. 另一条则是 Figma MCP 这样的接入路线，在不改变现有设计协作体系的前提下，把 Figma 里的组件、布局约束、设计令牌等结构化信息提供给 Claude Code、Cursor 这类 Agent，最近开始支持把代码逻辑或新需求反向带回 Figma，生成可编辑的设计层。

前者是在重做视觉上下文的生产方式，后者是在优化视觉上下文的接入方式。长期看，前一种方向可能更彻底；但短期内，设计工具本身仍然有很强的专业门槛和协作壁垒，很多复杂场景依然离不开专业设计师，也离不开 Figma/MasterGo 这类成熟工具承载评审、协作和交付流程。所以在可预见的阶段里，更现实的路径不是指望 Agent 直接取代设计工具，而是先把设计工具里的信息更好地整理成 Agent 真正可用的视觉上下文。

换句话说，问题已经不只是让 Agent “看见”设计稿，**而是视觉信息到底应该在哪里、以什么形式，被整理成可实现、可验证、可持续迭代的上下文。**

工具是做给 AI 用的

《The Complete Guide to Agentic Coding 2026》中有一条重要的设计原则：**Agent 工具需要与人类 API 不同的设计思维。** 工具要用自然格式（prose、markdown）而非复杂结构化格式，要提供充分的文档，要优雅地处理错误，要假设 Agent 可能会以意想不到的方式使用它们。

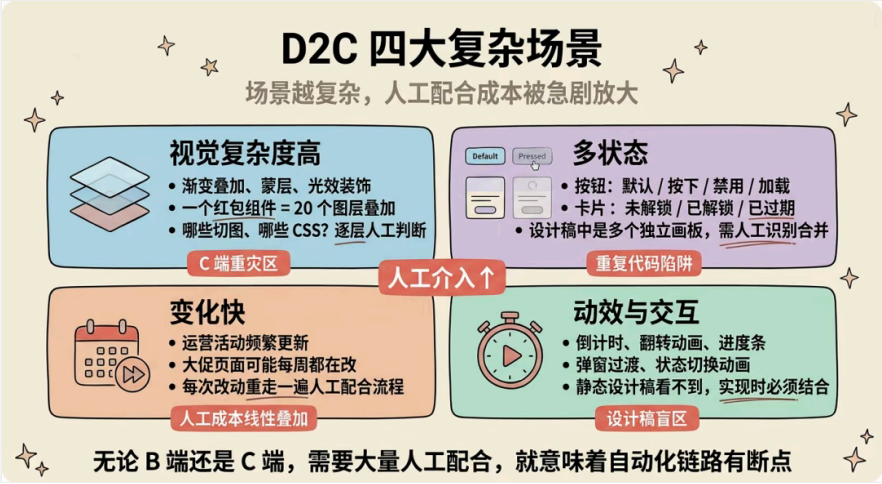
对视觉还原而言，这意味着：

- **设计稿是高度数字化的资产，应该能被 AI 很好地理解和使用**

- **不要脱离业务背景去进行纯粹的视觉还原，而是让 Agent 在拥有完整上下文的情况下按需进行**
- **工具是做给 AI 用的，不是做给人用的**

03 为什么需要新的视觉还原方式

D2C 的结构性矛盾



传统 D2C 平台在布局识别、组件匹配、样式提取等方面积累了大量工程经验。对于布局规整、组件标准化的场景，D2C 确实能显著提升效率。

但 D2C 的核心矛盾不是“能不能用”，而是**自动化链路中仍有大量环节依赖人工**：

- **图层整理与打标**：开发者需要手动打开设计工具，找到目标区域，选择相关图层——稍有遗漏就缺少关键信息，多选了又会引入噪声
- **切图与合图**：哪些该切图、哪些该用 CSS、哪些该合并，需要人工判断和手动标记
- **多状态识别**：同一组件的不同状态是多个独立画板，开发者需要自己识别并手动合并
- **一次性输出**：生成完代码后，设计稿信息的使命就结束了，发现还原度不够只能重新走一遍流程。而且开发者无法干预 D2C 平台的生码过程，甚至无法用 vibecoding 的方式对话式修正，只能拿到生成结果后人工微调

这些问题在标准化的 B 端场景中尚可控制 —— 布局规整、组件标准、状态简单。但当场景复杂度上升，人工成本会被急剧放大：

- **视觉复杂度高**：C 端设计师追求极致的视觉效果，大量使用渐变叠加、蒙层混合、PEN 路径蒙版、光效装饰。一个"红包组件"可能由 20 个图层叠加而成 —— 渐变背景、纹理叠加、光效装饰、弧形蒙版、文字、按钮，层层堆叠。哪些该用 CSS 实现、哪些该切图、哪些该合并成一张背景图，需要开发者逐层判断

- **多状态**：同一个按钮有默认态、按下态、禁用态、加载态；同一个卡片有未解锁、已解锁、已过期三种状态。设计稿中这些状态通常是多个独立画板，D2C 为每个画板独立生成代码，开发者需要自己识别"这三个画板其实是同一个按钮的三种状态"，然后手动合并成一个带条件渲染的组件
- **变化快**：运营活动频繁更新，一个大促页面可能每周都在改，每次改动都要重新走一遍人工配合流程
- **动效与交互**：倒计时、翻转动画、进度条、弹窗过渡——这些交互逻辑在静态设计稿中是看不到的，但实现时必须和视觉还原紧密结合

无论 B 端还是 C 端，"需要大量人工配合"这个事实本身，就意味着 D2C 的自动化链路存在结构性的断点。

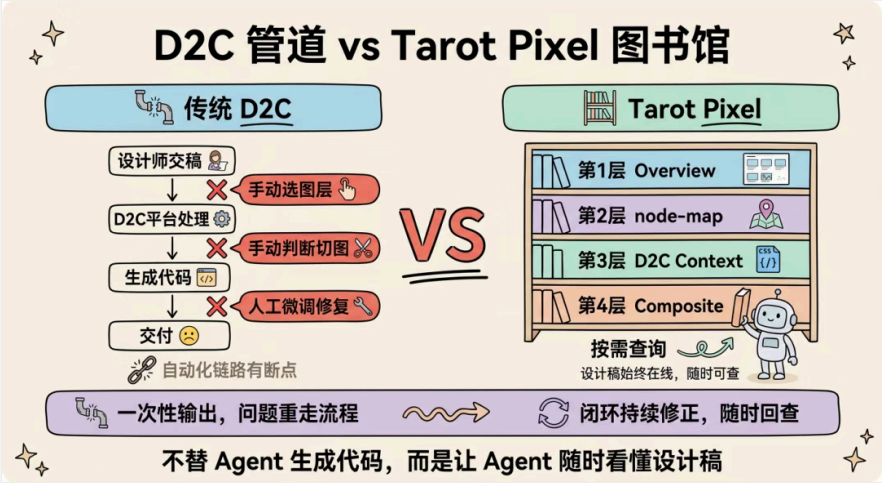
视觉还原不是孤立任务

上面讨论的是 D2C 流程内部的问题。但更根本的矛盾在于：**前端没有独立的 D2C 任务，**真正交付一个前端功能，视觉还原只是其中一个环节。它必须和交互逻辑、状态管理、接口联调放在同一套实现里完成。一个看似简单的"商品卡片"，实际上要处理：

- **与业务状态的耦合**：任务是否已完成、优惠券是否已过期、活动是否已结束 —— 这些状态决定了视觉表现（按钮置灰、标签变色、倒计时消失），但这些信息在设计稿中只是不同的画板，D2C 无法知道它们背后的业务含义
- **与交互逻辑的耦合**：弹窗弹出收起、Tab 切换内容区、状态变更时的过渡动画 —— 视觉元素不是静态展示，它们承载着交互行为。脱离这些上下文单独生成的 D2C 代码，只是一个空壳
- **与数据流的耦合**：价格从接口获取、库存实时变化、用户状态影响展示——这些动态数据决定了组件的最终渲染，但 D2C 只能看到设计稿中的静态占位

脱离业务上下文单独生成的代码，通常很难直接投入使用。这也是为什么我们需要换一种思路 —— **不替 Agent 生成代码，而是让拥有完整项目上下文的 Agent 自己来写。**

04 换一个思路——不建管道，建图书馆



Agent-Native 的设计逻辑

传统 D2C 试图建一条从设计稿到代码的完美管道。Tarot Pixel 换了个思路，出发点是第一章的核心判断：**既然 Agent 已经工作在“观察 → 规划 → 执行 → 反思”的自主循环里，工具应该顺应这种工作方式，而不是试图替代它。**

Agent 不像传统程序那样需要一次性拿到所有输入然后输出结果。它更像一个有经验的开发者：先浏览整体设计，确定要做什么；然后针对具体模块查看细节；写完代码后比对效果；发现问题再回去看设计稿。所以我们不建管道，建一个 AI 能查阅的“视觉参考系统”：

1. 将设计稿一次性导出为结构化的视觉预览（HTML 渲染 + 结构化数据），导出过程中自动完成降噪、蒙版处理、装饰图层标记
2. 在本地启动一个 Web Agent，提供按需查询的 REST API
3. Coding Agent 通过 Skill 文档知道有哪些 API 可用，自主决定查什么、怎么查、怎么用

关键在于：**设计稿不是输入，是参考资料。** 就像一个有经验的前端开发者会反复切回设计稿看细节，Coding Agent 调一次 API 获取一个信息点。需要按钮样式？查一下。需要间距？查一下。需要背景图？调合图 API 生成一张。不确定某个图层是什么？让 Chat API 的 LLM 判断。

代码生成完全交给 Coding Agent。 系统只负责“让 Coding Agent 看得懂设计稿”，具体怎么写代码、用什么技术栈、适配什么规范 —— 这些由 Coding Agent 结合项目上下文自行决定。这样做的好处是：同一套 Tarot Pixel 能力可以接入不同项目，最终写出的代码风格仍然是项目自己的，而不是平台模板化的输出。

这解决了什么

回到第二章提到的人工配合成本，看看 Tarot Pixel 如何尝试降低这些成本：

粒度控制 → 不再需要手动选图层。设计稿一次性导出，Coding Agent 通过 `overview` API 看到所有模块的预览图，自己定位到目标区域，自己决定需要哪些节点的信息。

切图 → 尽可能自动化。工程层自动分析哪些是装饰层、哪些是内容层。Coding Agent 看到 `[likely-decorative]` 标记，调 `composite` API 一键合成背景图。20 个装饰图层变成一张 PNG。

多状态 → Agent 辅助识别。通过模块 diff 和相似节点检测，系统能识别哪些模块是同一组件的不同状态。Coding Agent 根据此生成带条件渲染的单一组件，而不是多套重复代码。

IR 问题 → 换一种信息组织方式。没有大块 JSON 直接塞进上下文，信息按需获取，每次查询只返回必要的数据。

一次性输出 → 变成持续可对话。设计稿信息始终在线、随时可查。哪怕第一次还原度只有 80%，开发者对 Coding Agent 说“背景图的圆角不对，红包卡片缺了分割线”，Coding Agent 也可以自己回查、比对、修正。

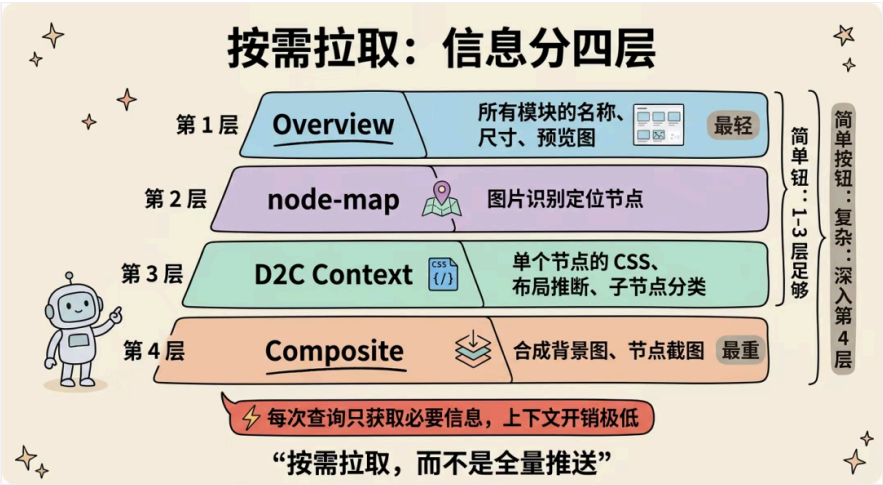
对复杂页面来说，一次实现不完美并不关键，关键是它已经处在一个可观察、可规划、可执行、可反思的闭环里。

渐进式上下文

设计稿的原始复杂性（复杂图层树、大量装饰节点、多状态画板）如果直接塞进上下文，会严重干扰 Agent 的编码决策。

Tarot Pixel 对这个问题的回答是：**按需拉取，而不是全量推送。** 且每一层信息都经过工程层的清洗和降噪，确保交到 Agent 手上的是干净、精确、可直接使用的信息。

传统 D2C 把整个 IR 塞进上下文；Tarot Pixel 让 Coding Agent 按需查询。信息是分层的，Coding Agent 按需逐层获取：



实现一个简单按钮，只需要第 1 层和第 2 层。实现一个复杂卡片，才需要深入到第 3、4 层。每次查询只获取必要信息，上下文开销极低。

Tarot Pixel 的所有 API 只传递信息：

API	输出	代码由谁写
`d2c-context`	CSS 属性、布局线索、图层分类	Coding Agent
`composite`	合成后的 PNG 图片 URL	Coding Agent 决定怎么用
`screenshot`	设计稿截图	Coding Agent 自己比对
`chat`	图层定位和切图建议	Coding Agent 根据此决策
`node-map`	图片识别定位结果	Coding Agent 据此导航

代码生成完全交给 Coding Agent——它本身就拥有项目的完整上下文，知道该用什么技术栈、什么样式规范、什么状态管理方案。这也是 Tarot Pixel 选择“只传递信息、不生成代码”的原因：项目上下文天然存在于 Coding Agent 中，让它自己做实现决策，比任何外部平台试图猜测项目规范都更准确。

05 Skill 驱动的 Agent 协作

Tarot Pixel 的 Skill 设计

上一篇文章讨论了 Specflow 如何通过外置 Agent 完成深度分析，然后把精简的任务上下文交给 Coding Agent。Tarot Pixel 的协作方式不同——它不是一个外置 Agent，而是**通过 Skill 文档赋予 Coding Agent 新的能力**。

Tarot Pixel 的 Skill 文档（`/skills/tarot-pixel/SKILL.md`）是一份

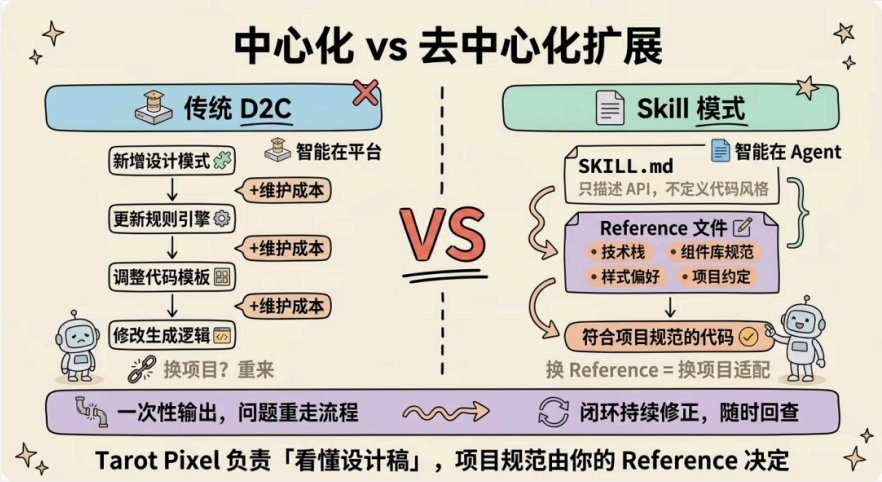
纯 API 参考文档：

- 告诉 Coding Agent 有 `overview`、`d2c-context`、`composite`、`screenshot`、`chat`、`node-map` 等 API
- 告诉 Coding Agent 每个 API 返回什么、什么场景下该用、什么场景下不该用
- 告诉 Coding Agent 几条通用原则（装饰优先切图、内容优先代码、所有数值必须来自 API）
- 提供标准工作流参考（定位 → 取证 → 决策），但不强制执行顺序

它不会规定代码风格，不会规定必须用什么组件库，也不会定义项目里的实现规范。 这些内容完全应该由项目自己的 Skill、README、组件文档或工程约束来决定。Tarot Pixel 只负责一件事：把视觉信息干净地交给 Coding Agent。

这就是 Skill 模式的生命力所在：**新增一种图层类型？只需要在工程层更新分类标签，Skill 不用改。新增一种布局模式？工程层输出新的布局推断文本，Skill 不用改。换了设计工具？只要输出同样格式的数据，Skill 和 Coding Agent 的行为完全不变。**

中心化扩展与去中心化扩展



传统 D2C 平台的扩展方式是：每支持一种新的设计模式，就需要更新规则引擎、调整代码模板、修改生成逻辑。这是一种**中心化的能力扩展** —— 所有智能都集中在平台侧。这种方式的好处是可控性强，但维护成本也随着覆盖场景的增加而线性增长。

Skill 模式提供了另一种可能：工程层提供新的数据标签或 API，Coding Agent 自己学会怎么用。这是一种**去中心化的能力扩展**——智能分布在 Agent 侧。

两种方式各有适用场景。但在 Agent 能力快速进化的今天，去中心化扩展有一个天然优势：**模型变聪明了，它对 Skill 文档的理解和运用自然就更好了，系统不需要为此做任何改动。** 工程层只需要持续提供更精确的数据和标签，Agent 侧的能力提升会自动转化为更好的还原效果。

不追求一次性完美，追求可迭代

“一次性完美生成”是 D2C 领域长期追求的目标，也是一个很有价值的方向。但在实践中，“完美”的定义往往取决于项目上下文、业务需求、交互逻辑——这些信息很难在一次性的生成流程中被完整覆盖。

Tarot Pixel 在此基础上增加了一个补充目标：**可对话迭代。**

哪怕第一次生成的代码还原度只有 80%，开发者也不需要教它“该怎么改”，只需要指出“哪里不像/不对”。Coding Agent 会自己回去查设计稿、理解修正意图、补充取证并完成修改。

为正在到来的 Agent 生态而设计

站在 2026 年初回看，有几个趋势正在加速，它们验证了 Tarot Pixel 的设计方向，也指向了更大的可能性：

- **模型能力的持续跃升。** 即便当下的模型还不能 100% 一次性完美还原，半年后能做到的模型会有很多。Tarot Pixel 的系统不需要为此做任何改动——因为我们从来没有把“代码生成”这个最依赖模型能力的环节绑定在自己身上。

- **Browser Use 能力的普及。** 越来越多的 Coding Agent 内置了 Browser Use 能力——能自主打开浏览器、截图、交互。这对 Tarot Pixel 来说是一个天然的增强：Coding Agent 可以截取自己实现的页面，与设计稿截图对比，通过 `node-map` 或 `chat` API 定位差异，形成更紧密的视觉反馈闭环。
- **多模态理解的进化方向。** 模型的视觉理解能力在快速提升，但这并不意味着“直接把设计稿截图丢给 AI”就够了。模型能“看懂”一张图，不等于它能从中提取像素级精确的 CSS 值、判断哪些图层该合并切图、识别多个画板间的状态关系。**工程层的精确数据提取和 AI 层的语义理解是互补的，不是替代的。**
- **Skill + REST API 的通用性。** 从最初简单的 `SKILL.md` 到现在社区涌现的各种 Skill 规范，“用声明式文档描述工具能力”正在成为 Agent 生态的基础设施。Tarot Pixel 的设计——纯 API 参考、不绑定流程、不定义项目规范——恰好符合这个趋势。REST API 比任何特定的 Agent 协议都更通用：无论是 Coding Agent、Claude Code 还是其他 Coding Agent，只要能发 HTTP 请求，就能直接调用 Tarot Pixel 的服务。

这些趋势共同指向一个判断：**为 Agent 提供干净的信息和工具，比替 Agent 做决定更有长期价值。**

我没有做 Agent，
我做了一个为 Agent 服务的系统

Agent-native 的设计理念

这是做 Tarot Pixel 过程中最核心的思考。

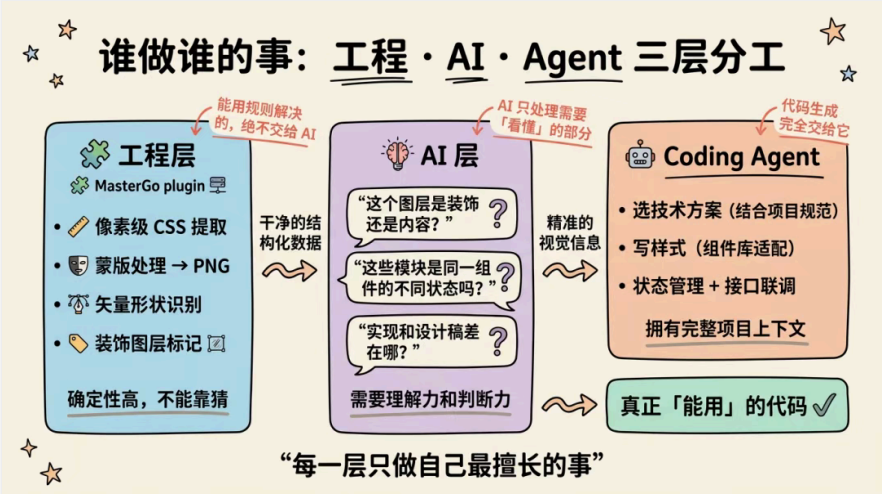
这套系统没有复杂的 RAG、没有微调的专有模型、没有多 Agent 协作框架。从技术栈上看，它甚至不像一个“AI 项目”——MasterGo 插件是纯工程（TypeScript + React），Viewer Server 是标准的 HTTP 服务（Bun + Vite），前端是普通的 React 19 应用。

但它的核心设计理念是 **Agent-native** 的：

Claude Code/Cursor/Qoder 本身就是 Agent。 它拥有项目代码库的完整上下文，拥有编码能力，拥有自主决策能力。我不需要再造一个 Agent，我需要的是**为这个已经存在的、能力强大的 Agent 提供它缺失的能力**——稳定使用设计稿信息的能力。

所以 Tarot Pixel 的定位不是“一个 D2C 平台”，而是“Coding Agent 的视觉感知层”。

工程 + AI 的边界



这种设计的好处是**每一层只做自己最擅长的事**：

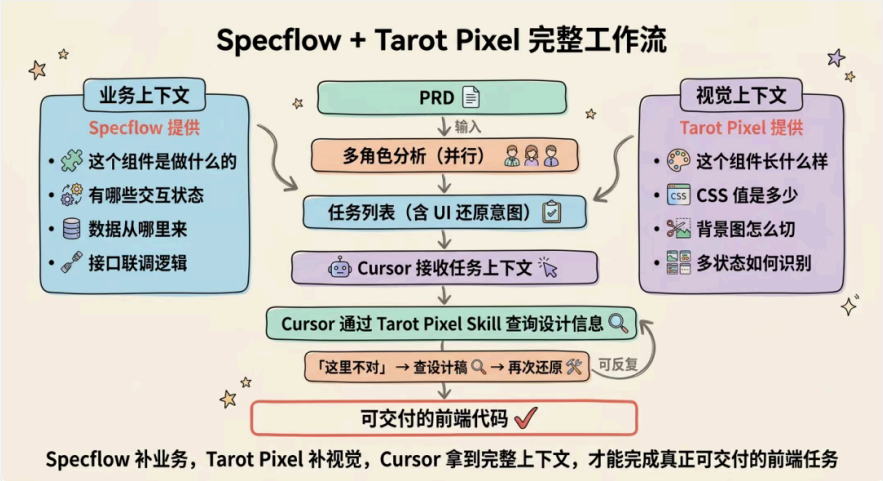
- **工程做工程擅长的事**：像素级精确的 CSS 提取、资源导出、蒙版处理、布局推断——这些靠规则和算法，确定性高，不能出错
- **AI 做 AI 擅长的事**：理解“这个图层是装饰还是内容”“这些模块是同一组件的不同状态”“实现和设计稿差在哪”——这些需要理解能力
- **Coding Agent 做 Coding Agent 擅长的事**：结合项目代码库，用正确的组件、正确的规范、正确的架构写出能用的代码

能用工程确定性解决的，绝不交给 AI。 AI 只处理需要理解力和判断力的部分。

层	职责	为什么是这样
工程（MasterGo 插件）	精确的数据提取和格式转换	像素级 CSS、资源导出、蒙版处理——需要确定性，不能靠 AI 猜
工程（Viewer Server）	数据加工和 API 暴露	布局推断、图层分类、复合图合成——算法可解决的结构化分析
AI（Chat LLM）	视觉语义理解和定位	模块定位、状态识别、图层角色判断、图片识别——需要“看”和“理解”的任务
AI（Coding Agent）	结合上下文的代码生成	选技术方案、写样式规范、管理状态——需要项目上下文和编码能力

与 Specflow 的协作模式

在完整的工作流中，Specflow 和 Tarot Pixel 各司其职：



Specflow 提供的是**业务上下文**——这个组件是做什么的、有哪些交互状态、数据从哪来。Tarot Pixel 提供的是**视觉上下文**——这个组件长什么样、CSS 值是多少、背景图怎么切。两者结合，Coding Agent 才能生成真正“能用”的代码，而不是一个空壳。

这也呼应了前文的判断：前端没有独立的 D2C 任务。 Specflow 负责补齐业务和实现上下文，Tarot Pixel 负责补上视觉信息的缺口，二者合在一起，才能完成真正可交付的前端实现。

更进一步看，**这种模式也可能会改变前后端原有的协作边界**。因为当设计稿信息以这种方式接入 Coding Agent 后，视觉还原本身不再要求操作者必须懂图层、会切图、熟悉 CSS。哪怕是服务端同学，只要能指出“哪里不像”“哪里不对”，Coding Agent 就能自己回去查设计稿并修正页面。

07 系统架构与实现

前面几章讨论了设计理念和协作模式，这一章简要介绍 Tarot Pixel 的系统架构和核心实现。

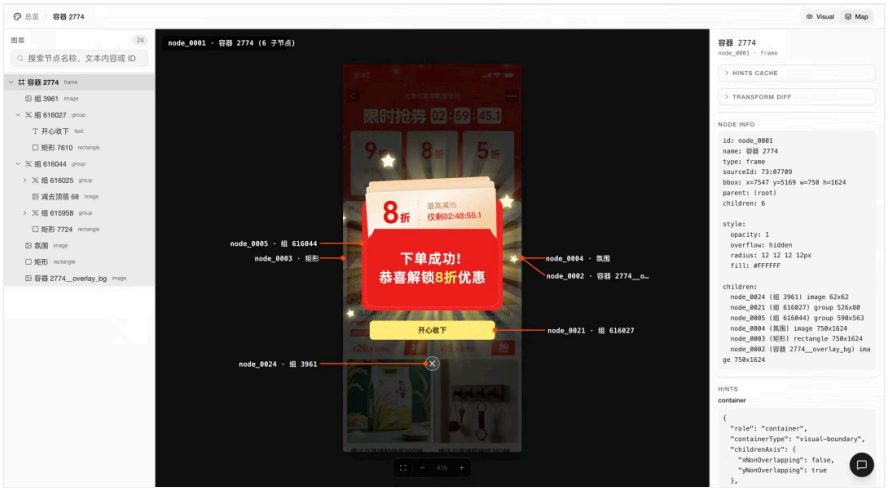
系统模块

Tarot Pixel 由四个核心模块组成：

CLI 命令行工具：基于 TUI 的交互式界面，支持视觉稿管理、模型配置、服务启动、Skill 安装等操作。开发者不需要记忆命令参数，通过菜单即可完成所有配置。

Viewer 可视化中间层：将设计稿渲染为一比一复刻的 Web 界面，同时服务于三个场景：

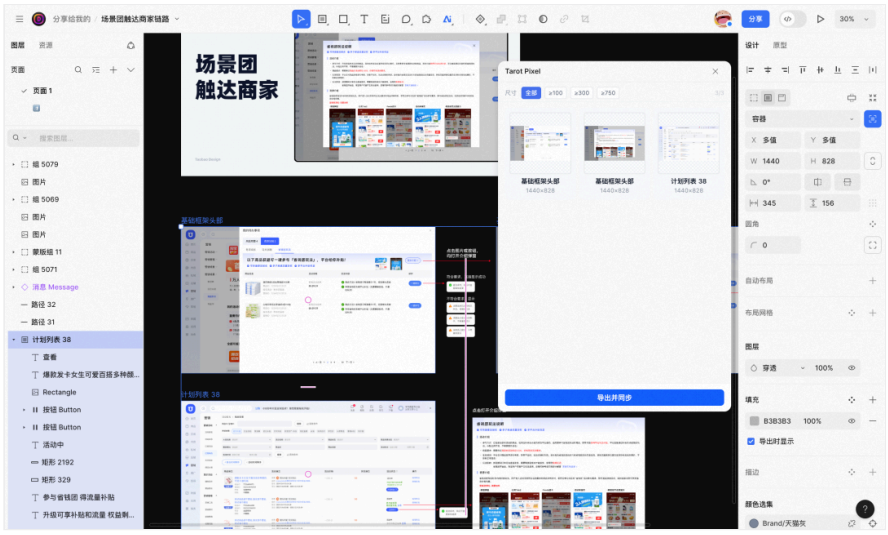
- 开发者预览：图层树 + 画布 + 属性面板，无需打开设计工具即可浏览设计稿
- Agent Browser Use：提供 Web Agent API，Coding Agent 可直接“看到”设计稿
- Map 模式：在视觉稿上叠加节点边界标注，配合 node-map API 实现视觉元素到节点 ID 的快速映射



API 服务：本地运行的 HTTP 服务，提供 overview、d2c-context、composite、screenshot、chat、node-map 等接口。设计原则是按需拉取、分层获取、幂等无状态。

内置 Agent 系统：通过 Chat API 暴露，作为 Coding Agent 的“视觉顾问”，处理图层定位、切图决策、还原度比对等需要视觉理解的任务，分担 Coding Agent 的上下文压力。

MasterGo 插件：数据层的核心组件，负责设计稿的解析和降噪处理。与传统 D2C 平台一个模块一个模块导出不同，**插件支持整个设计稿一次性导出，多个模块并行处理、同时预览**。开发者不需要小心翼翼地整理图层结构、选择需要的部分——直接导出整个页面，Coding Agent 自己会定位到需要的那一块。



插件将设计师的原始图层树转换为结构化数据，过程中自动完成：

- 蒙版翻译：简单蒙版（矩形/椭圆）转 CSS `overflow: hidden` + `border-radius`；复杂蒙版（PEN 路径、布尔运算、渐变蒙版）自动导出为 PNG，文字保持动态可编辑
- 矢量形状识别：PEN 工具画的圆通过 SVG 路径分析识别，转为 CSS `border-radius: 50%` 而不是导出 SVG 图片

- 装饰图层标记：纯装饰组合（渐变、光效、阴影叠加）自动检测并标记为 `[likely-decorative]`，包括 Ghost 节点、背景节点、分隔线等分类
- 精度控制：字重修正、透明度去重、数值四舍五入
- 资源自动导出：蒙版强制 PNG，简单矢量用 SVG，图片填充用 PNG

这些处理的本质是：**把不适合直接进入上下文的复杂性提前清洗掉。** 无论设计师用了多复杂的蒙版操作，开发者无需理解，系统自动处理。

信息密度控制

降噪不仅发生在导出阶段，也贯穿于查询和合图阶段。

查询阶段：D2C Context 是经过精心编排的结构化文本，而不是原始数据 dump。关键设计包括：

- 节点分类标签（`[likely-decorative]`、`[keep-text]`）告诉 Coding Agent 每个子节点的角色
- 布局推断（`Layout: flex column gap 60px`）工程层已分析好布局方式
- 深度控制：一次请求只返回一个节点及其直接子节点，避免上下文过大

合图阶段：设计师的一个“卡片”可能由 20 个图层叠加。工程层自动分析装饰层与内容层，装饰层通过 Playwright 截图合并为一张背景 PNG。核心原则：**合并粒度是视觉单元，不是节点。**

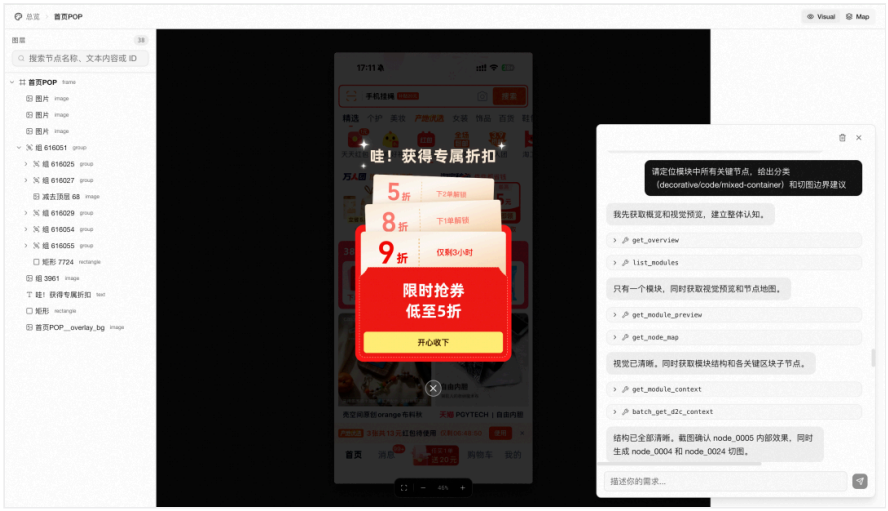
辅助决策与定位

在实际使用中，我们发现了一个值得关注的问题：**Coding Agent 在单窗口下的上下文是有限的。** 一个完整的前端任务中，需求描述和任务背景先占据一部分上下文，定位相关文件和理解项目结构再占据一部分，留给视觉还原的空间已经被压缩了。如果图层定位、切图决策、视觉语义判断这些工作还要由 Coding Agent 自己完成，它需要反复调用 API、逐层遍历节点树——留给真正编码的空间就更少了。

为此，Tarot Pixel 将部分需要视觉理解的工作交给了自己的 **Chat API**。Chat API 背后是一个独立的 LLM Agent，拥有自己的上下文窗口和工具调用能力。它可以：

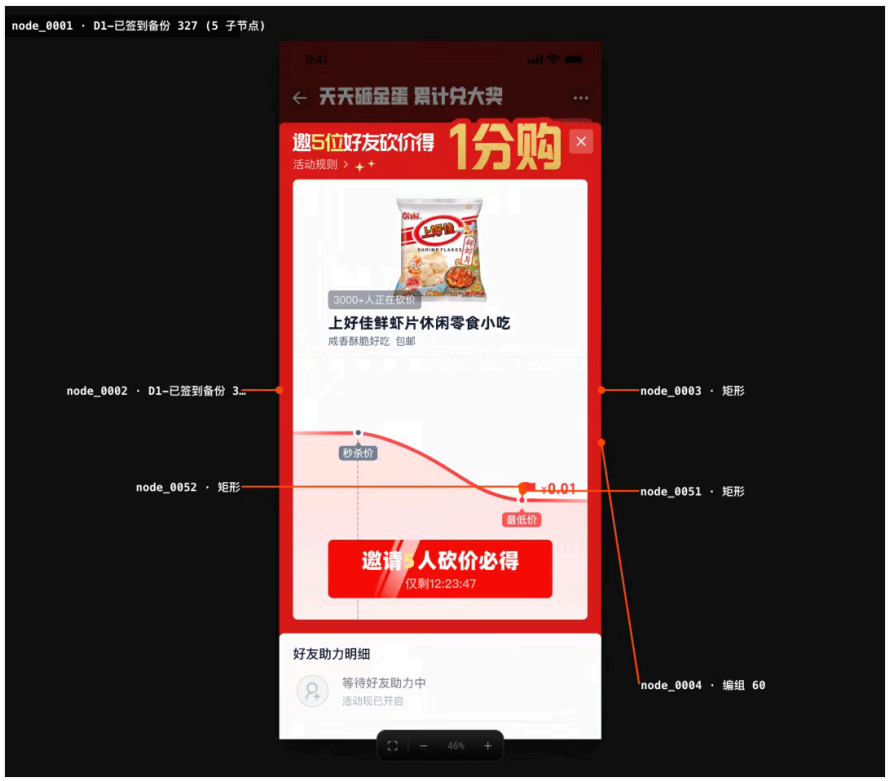
- **图层定位**：开发者或 Coding Agent 描述"我要找红包组件的按钮"，Chat API 自己遍历节点树、结合截图做视觉匹配，返回精确的节点 ID
- **切图决策**：判断一组图层应该合并切图还是用 CSS 实现

这种设计的本质是**用一个专门的视觉 Agent 分担 Coding Agent 的上下文压力**。Coding Agent 只需要发一次请求、拿到结论，而不是自己做大量的中间推理。



对话过程由 Cursor 完成

此外，我们还提供了 `node-map` API——支持通过图片识别来定位节点。Coding Agent 可以把设计稿截图的某个区域发给 `node-map`，API 通过视觉匹配返回对应的节点信息。这在 Coding Agent 已经普遍内置 Browser Use 能力的今天尤其有用：Coding Agent 可以截取自己实现的页面截图，与设计稿截图对比，通过 `node-map` 快速定位到需要修正的节点，形成 **screenshot → 识别 → 定位 → 修正** 的闭环。



与 MasterGo MCP 的差异

当前大部分使用 MasterGo MCP 的做法是：在 MasterGo 中手动选择容器获取 ID，粘贴到 AI 对话中，MCP 调用 API 获取结构化数据，然后 AI 生成代码。

Tarot Pixel 与这种做法有几个本质差异：

人工预处理 vs 自动化。 MasterGo MCP 需要开发者手动选择容器、处理分组、识别多状态、判断切图——这仍然是“人做预处理，AI 做翻译”的模式。Tarot Pixel 支持整个设计稿一次性导出，多个模块并行处理、同时预览。

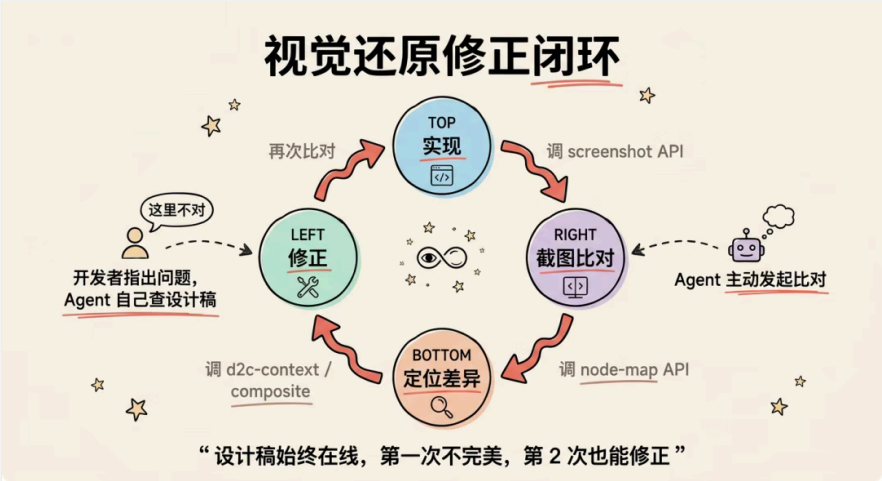
全量返回 vs 按需查询。 MasterGo MCP 返回的是选中节点的全量 JSON 数据，一次性注入上下文。Tarot Pixel 提供分层 API，Coding Agent 可以先查 overview 了解整体结构，再按需查询具体节点的 d2c-context，避免上下文过载。

缺乏辅助手段 vs 完整工具链。 MasterGo MCP 只提供数据获取，没有截图、合图、节点定位、还原度比对等辅助能力。Tarot Pixel 提供完整的 20+ API，包括 Chat Agent 的视觉分析、node-map 的图片识别定位、composite 的自动合图等。

一次性交付 vs 持续在线。 MasterGo MCP 获取数据后，设计稿信息的使命就结束了。Tarot Pixel 的设计稿信息始终在线，实现、比对、修正都可以随时回查。

08 修正闭环与干预成本

修正闭环



Coding Agent 实现完代码后，视觉还原的修正可以有两种方式发生：

开发者提示修正：开发者发现某处还原不对，只需说一句“这里的间距不对”或“这个背景色不对”，Agent 就能自己回查设计稿、自己修正——不需要开发者告诉它具体该改成什么值。

自主修正：Coding Agent 也可以主动发起比对：

1. 调截图 API 拿到设计稿原图，与自己的实现对比
2. 调 `d2c-context` 查看遗漏的细节，或调 `composite` 获取合成图修正背景
3. 通过 `node-map` 用图片识别快速定位到需要修正的节点

无论哪种方式，都形成了一个闭环：实现 → 比对 → 修正 → 再比对。这个闭环不要求第一次必须完美——只要设计稿信息始终在线，Agent 随时可以回去查和改。

随着 Coding 普遍内置 Browser Use 能力，这个闭环会变得更加自然：Coding Agent 可以直接打开浏览器查看自己的实现效果，截图后与设计稿对比，通过 REST API 获取修正所需的精确数据。**Screenshot + REST API 的组合，让视觉反馈从“人工肉眼比对”变成了 Agent 可以自主完成的结构化流程。**

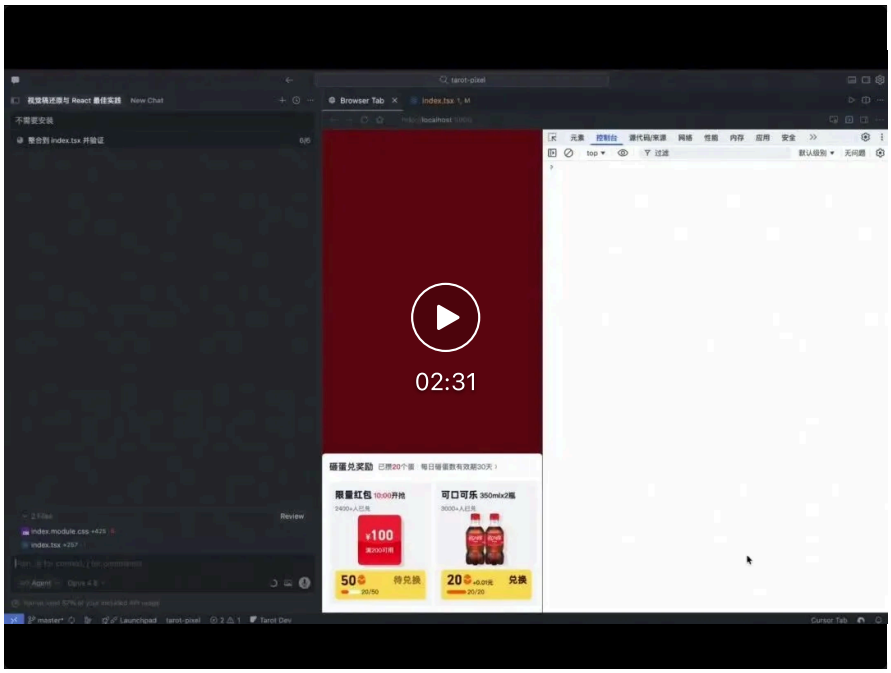
真正的效率指标：人工干预次数

衡量 AI Coding 是否真正有效，光看“AI 代码采纳率”是不够的，更关键的指标其实是**人工干预次数**：从需求到交付，开发者需要介入多少次。

Specflow 和 Tarot Pixel 做的是同一件事：一步步降低这个数字。Specflow 减少的是需求拆解和任务编排的干预；Tarot Pixel 减少的是视觉还原和修正环节的干预。传统 Vibecoding 是“人告诉 AI 怎么做”，Tarot Pixel 是“人指出问题，AI 自己查资料自己修”——干预成本完全不同。

设计稿信息持续在线，意味着无论首次实现还是后续修正，Agent 都能回到同一个信息源。设计稿不是一次性输入，而是随时可查的参考源。

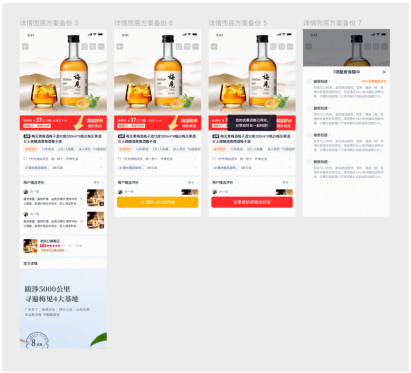
文章开始的演示视频里，其实首次生成的效果仍然有一些问题；下面这个视频展示的，就是 Coding Agent 在同一份设计稿上下文里继续完成二次修正的过程。

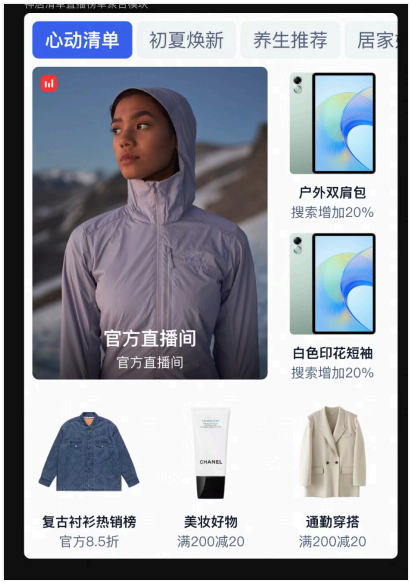


效果展示

以下是 Tarot Pixel 在实际场景中的还原效果。左侧为设计稿，右侧为 Coding Agent 借助 Tarot Pixel 生成的实际页面。**以下均为 Coding Agent 一次完成，人工干预前的效果。**

> 运行环境：Cursor + Sonnet 4.6 及以上、Qoder + 性能及以上。部分案例基于塔罗 Specflow 生成的业务需求描述，部分案例仅测试视觉还原度。

视觉稿	还原效果
	


心动清单
初夏焕新
养生推荐
官方直播间
官方直播间
户外双肩包
搜索增加20%
白色印花短袖
搜索增加20%
复古衬衫热销榜
官方8.5折
美妆好物
满200减20
通勤穿搭
满200减20

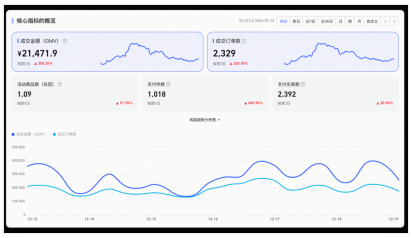
心动清单
初夏焕新
养生推荐
官方直播间
官方直播间
已售1000万件
户外双肩包
搜索增加20%
已售1000万件
白色印花短袖
搜索增加20%
已售1000万件
复古衬衫热销榜
官方8.5折
已售1000万件
美妆好物
满200减20
已售1000万件
通勤穿搭
满200减20

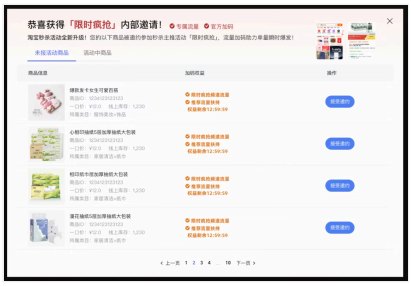
复古球鞋
奥运观赛
全民健身
复古球鞋热卖榜
33万人已种草
心动省钱好物来咯，超值性价比更划算
Top1
Top2
adidas旗舰店
Top3
NIKE旗舰店
Vans旗舰店
查看更多榜单 >

复古球鞋
奥运观赛
全民健身
复古球鞋热卖榜
33万人已种草
心动省钱好物来咯，超值性价比更划算
Top1
已售1000万件
Top2
已售1000万件
adidas旗舰店
Top3
已售1000万件
NIKE旗舰店
Vans旗舰店
查看更多榜单 >

618冲刺 仅限今日
规则
下一单就返 100元
02:24:36后任务结束
¥2 满2可用 今日可用
¥100 红包组合 下一单可得
仅限今日
规则
下一单就返 100元
02:24:36后任务结束
¥2 满2.01可用 今日可用
¥2 满2.01可用 今日可用
¥100 红包组合 下一单可得
618冲刺 仅限今日
规则
下一单就返 100元
02:24:36后任务结束
¥2 满2可用 今日可用
¥2 满2可用 今日可用
¥2 满2可用 今日可用
¥100 红包组合 下一单可得

买一单就返 100元
02:24:36后任务结束
¥2 满2可用 今日可用
¥100 红包组合 下一单可得
买一单就返 100元
02:24:36后任务结束
¥2 满2.01可用 今日可用
¥2 满2.01可用 今日可用
¥ 红包组合 下一单可得
买一单就返 100元
02:24:36后任务结束
¥2 满2可用 今日可用
¥2 满2可用 今日可用
¥2 满2可用 今日可用
¥ 红包组合 下一单可得
买一单就返 100元
02:24:36后任务结束
¥2 满2可用 今日可用
¥2 满2可用 已使用
待解锁 明天可用
¥ 红包组合 下一单可得





理想目标

让前端开发者在 AI 辅助开发中完全不需要手动处理设计稿。 不需要手动选图层、不需要手动切图、不需要手动标记多状态、不需要手动输入 IR。设计师交付设计稿，开发者对 Coding Agent 说“实现这个页面”，剩下的全自动。

同时，这种能力并不只属于前端。当视觉还原被重构为“指出问题 → Agent 自己查设计稿并修正”的模式后，哪怕是不懂 CSS、不熟悉图层结构的服务端同学，也可以通过自然语言参与高精度的页面修正。因为他不需要亲自翻译设计稿，只需要指出“哪里不对”。

当前进展

- **MasterGo 插件**：完成设计稿解析、自动蒙版处理、PEN 形状识别、复合图层标记、资源导出
- **Viewer Server**：完成全套 REST API（overview、d2c-context、composite、screenshot、chat、node-map 等 20+ 接口）
- **Node-Map**：支持图片识别定位节点，结合 Browser Use 能力形成视觉反馈闭环
- **Skill 文档**：适配 Cursor 和 Qoder，纯 API 驱动，不定义项目代码风格
- **CLI 工具**：一键启动本地服务，TUI 界面视觉稿一键切换
- **Chat AI Agent**：内置独立 Agent 支持视觉分析、节点定位、切图决策、还原度比对，内置 20+ 工具调用能力
- **降噪体系**：节点分类（decorative/text/container/interactive）、装饰检测、Ghost 节点识别、布局推断、信息密度控制
- **合图体系**：单父节点合图（composite）、多节点合图（multi-composite）、include/exclude 精确控制
- **能力测评**：目前收集了 34 个覆盖 BC 端典型场景的视觉稿做测试验证，完整流程验证完成 24 个，涉及从 MasterGo 导出到中间态视觉稿的工程性问题已全部解决，涉及编程 Agent 查询使用中间态视觉稿做 UI 真实还原的大部分问题都已解决（定位慢、图片合成不准确持续优化中），过程中发现 UI 真实还原性问题一般化 1-3 次人工对话引导都可以完成，相较于之前 D2C 的流程人工介入程度极低。

仍在解决的问题

当前方案还有一些明确的不足：

- **图层定位效率**：在复杂设计稿中，Coding Agent 自己遍历节点树定位目标图层的速度还不够快。目前通过 Chat API 和 node-map API 分担定位工作来缓解，但仍有优化空间
- **切图准确性**：装饰层与内容层的自动分类在大多数场景下有效，但在一些边界情况（如半透明装饰与内容重叠）下仍需人工确认
- **还原度**：一般场景可一次性高质量还原，复杂模块需要 1-3 轮额外对话微调
- **多状态识别**：基础能力已具备（模块 diff、相似节点检测），准确率持续优化中

这套方案对模型的视觉理解、长上下文处理、工具调用稳定性，以及多轮任务推进能力都有较高要求。模型能力不够时，即使上下文给得足够完整，也可能在定位、判断和连续修正上出现明显退化。

将来计划

- **更智能的降噪**：引入更多启发式规则和视觉语义分析，进一步减少 AI 需要处理的上下文量
- **更快速的图层定位**：让 Coding Agent 在复杂设计稿中能更快定位到目标图层
- **等待模型进化**：随着模型的视觉理解和代码生成能力持续提升，系统的还原度将自然提高——这是架构设计的前瞻性红利

10 总结

1. **D2C 的核心问题不是不能生成代码，而是自动化链路中仍有大量环节依赖人工配合。**
2. **当 Coding Agent 已经拥有业务上下文和项目上下文时，更合理的做法不是替它生成代码，而是为它提供干净、可查询、持续在线的视觉信息。**
3. **Tarot Pixel 不追求一次性完美生成，而是把设计稿接入 Agent 的观察—规划—执行—反思闭环，让页面能够持续修正。**
4. **无论是 Specflow 还是 Tarot Pixel，本质上都在做同一件事：为 Coding Agent 补足缺失能力，同时减少无效上下文干扰。**

说到底，它们遵循的是同一个朴素的原则：**给模型足够的上下文，减少对 Agent 的干扰。**

11 团队介绍

本文作者木偶，来自淘天集团–用户场景营销技术团队。一支专注于探索AI等前沿技术与营销业务技术的融合，深度结合用户场景与营销业务的技术团队。依托大淘宝丰富的用户生态和多元化的消费场景，致力于通过技术创新提升用户体验，优化个性化营销能力，助力业务持续增长。通过AI驱动的精准推荐、场景化表达与动态策略调控，我们为用户创造更自然、更智能的购物旅程，为营销业务提供高效、敏捷的技术支撑，助力淘宝构建以用户为中心的全域营销技术体系。我们坚信技术是连接用户与价值的桥梁，持续探索创新边界，让营销更懂用户，用技术点亮每一个关键用户体验瞬间。

大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 4342 修改于2026年6月24日

留言

写留言

还没有留言，来写首评